

若依微服务项目部署流程

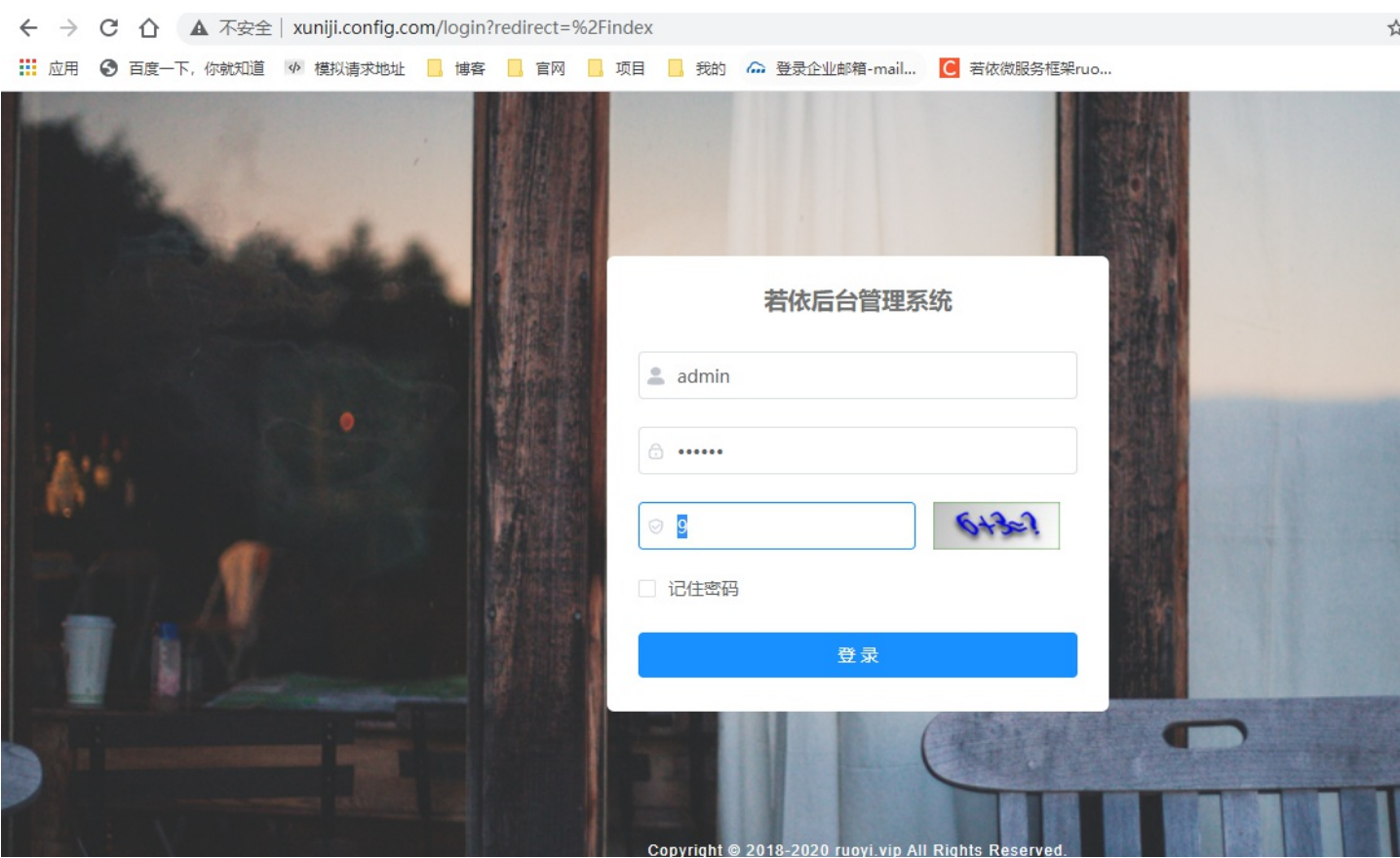
一.服务器说明

服务器一

序号	域名	ip	作用
1	xuniji.config.com	192.168.220.128	提供redis,nacos,mysql ,nginx服务， 前端代码也是部署到该服务器
2	xuniji.server.com	192.168.220.129	服务部署的机器， supervisor 进行项目线程统一管理

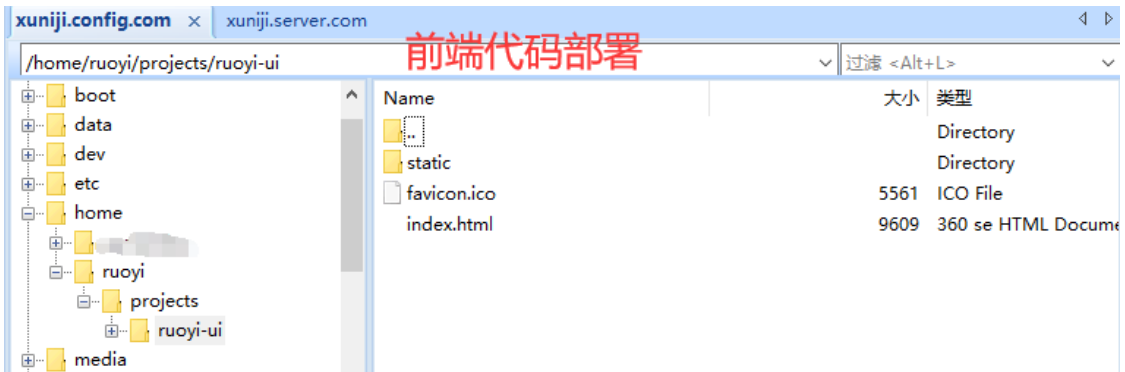
二.成品展示

1.页面展示

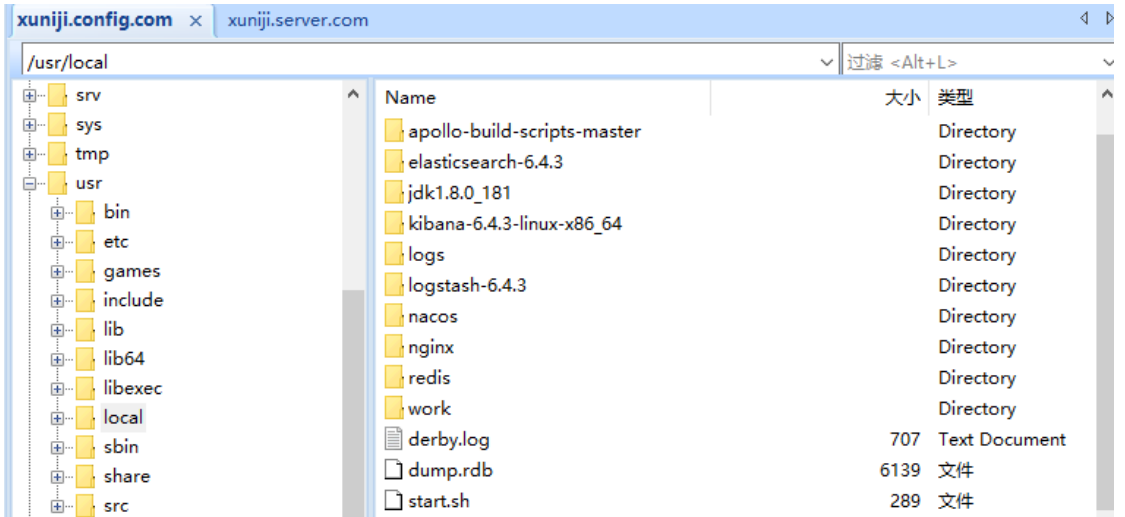




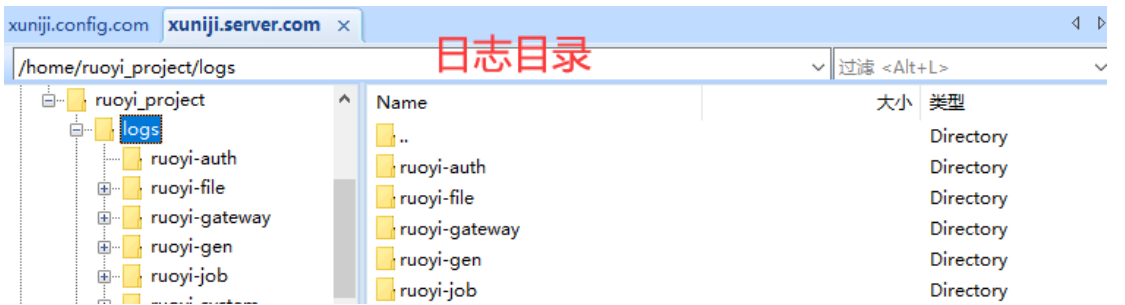
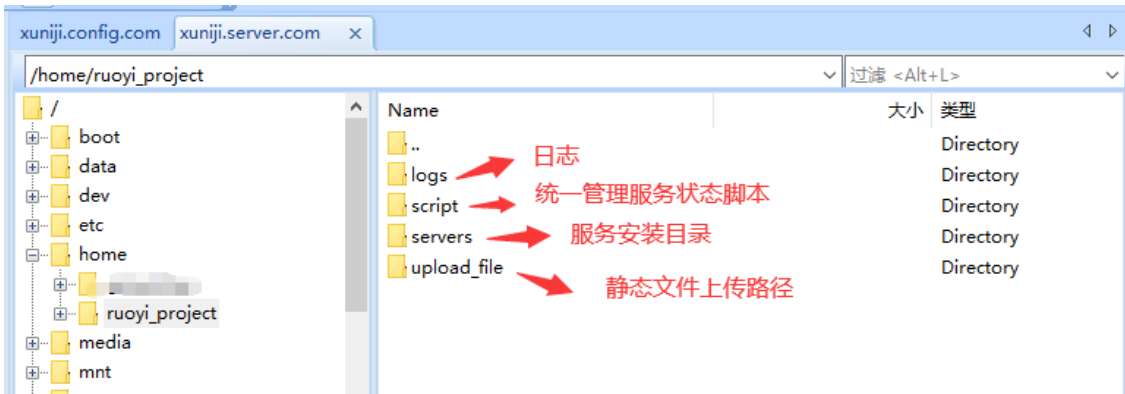
2.config机器前端代码部署路径



3.config机器中间件安装路径(只用到了redis, nginx, nacos其余的请忽略,start.sh是统一启动的shell脚本)



4.后端代码部署路径



ruoyi-system	ruoyi-system	Directory
ruoyi-test	ruoyi-test	Directory
ruoyi-visual-monitor	ruoyi-visual-monitor	Directory
script		

xuniji.config.com xuniji.server.com

/home/ruoyi_project/script

Name	大小	类型
..		Directory
shell_script		Directory
supervisor		Directory

1.shell脚本管理各个服务

2.supervisor守护进程管理哥哥服务(推荐)

xuniji.config.com xuniji.server.com

/home/ruoyi_project/servers/auth

Name	大小	类型
..		Directory
config		Directory
lib		Directory
ruoyi-auth-2.3.0.jar	163196	Executable Jar File

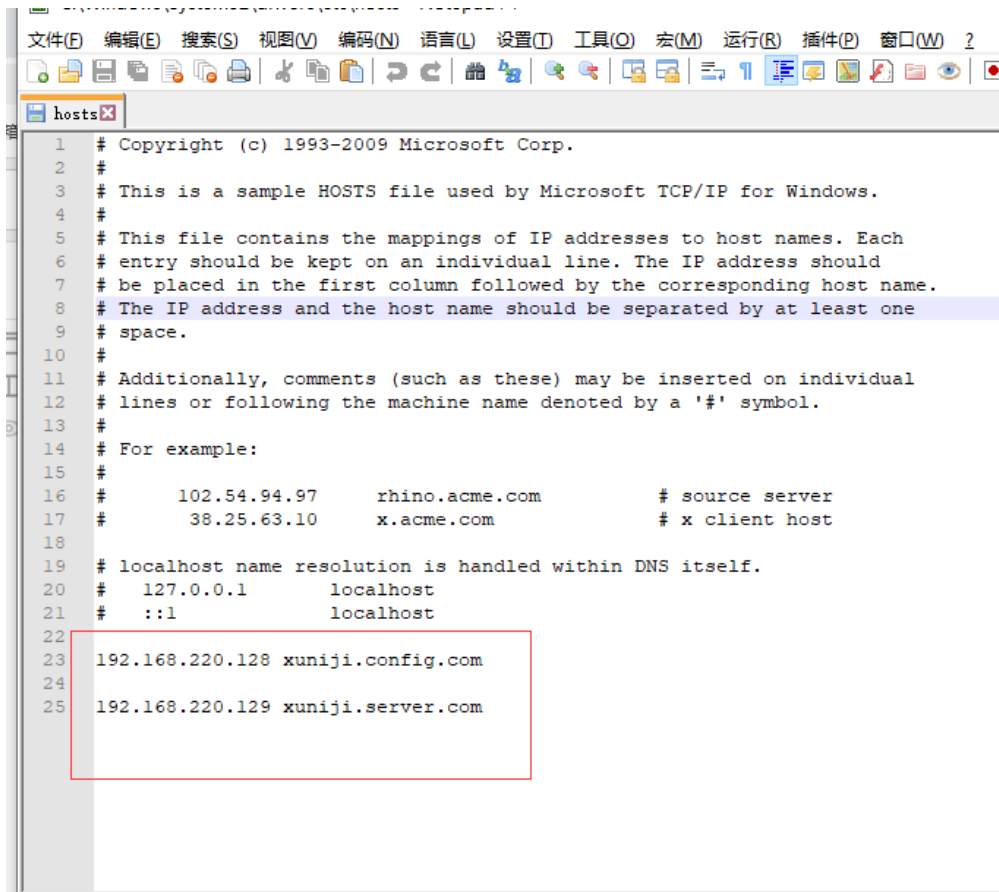
配置文件

依赖jar包分离

服务代码jar包

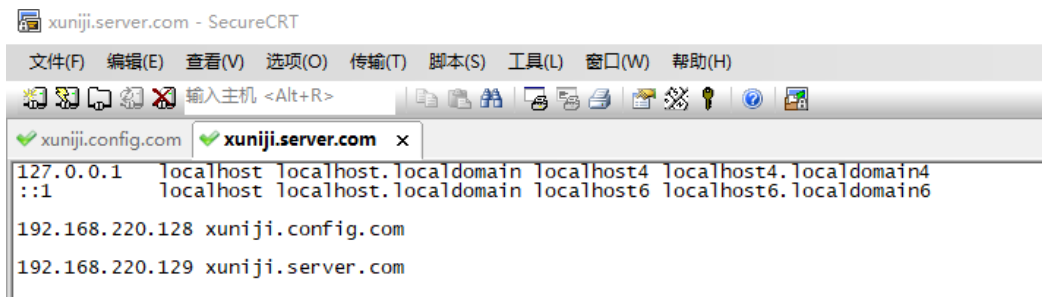
三.前期准备

1.由于我是在虚拟机上部署的，并没有真正的域名所以需要修改本机的C:\Windows\System32\drivers\etc下的hosts文件



```
1 # Copyright (c) 1993-2009 Microsoft Corp.
2 #
3 # This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
4 #
5 # This file contains the mappings of IP addresses to host names. Each
6 # entry should be kept on an individual line. The IP address should
7 # be placed in the first column followed by the corresponding host name.
8 # The IP address and the host name should be separated by at least one
9 # space.
10 #
11 # Additionally, comments (such as these) may be inserted on individual
12 # lines or following the machine name denoted by a '#' symbol.
13 #
14 # For example:
15 #
16 #         102.54.94.97       rhino.acme.com           # source server
17 #         38.25.63.10      x.acme.com               # x client host
18 #
19 # localhost name resolution is handled within DNS itself.
20 #   127.0.0.1      localhost
21 #   ::1            localhost
22
23 192.168.220.128 xuniji.config.com
24
25 192.168.220.129 xuniji.server.com
```

2.两台服务器的/etc下的hosts文件也需要修改



```
xuniji.server.com - SecureCRT
文件(F) 编辑(E) 查看(V) 选项(O) 传输(T) 脚本(S) 工具(L) 窗口(W) 帮助(H)
输入主机 <Alt+R>
xuniji.config.com xuniji.server.com x
127.0.0.1 localhost localhost.localhost localhost4 localhost4.localhost4
::1 localhost localhost.localhost localhost6 localhost6.localhost6
192.168.220.128 xuniji.config.com
192.168.220.129 xuniji.server.com
```

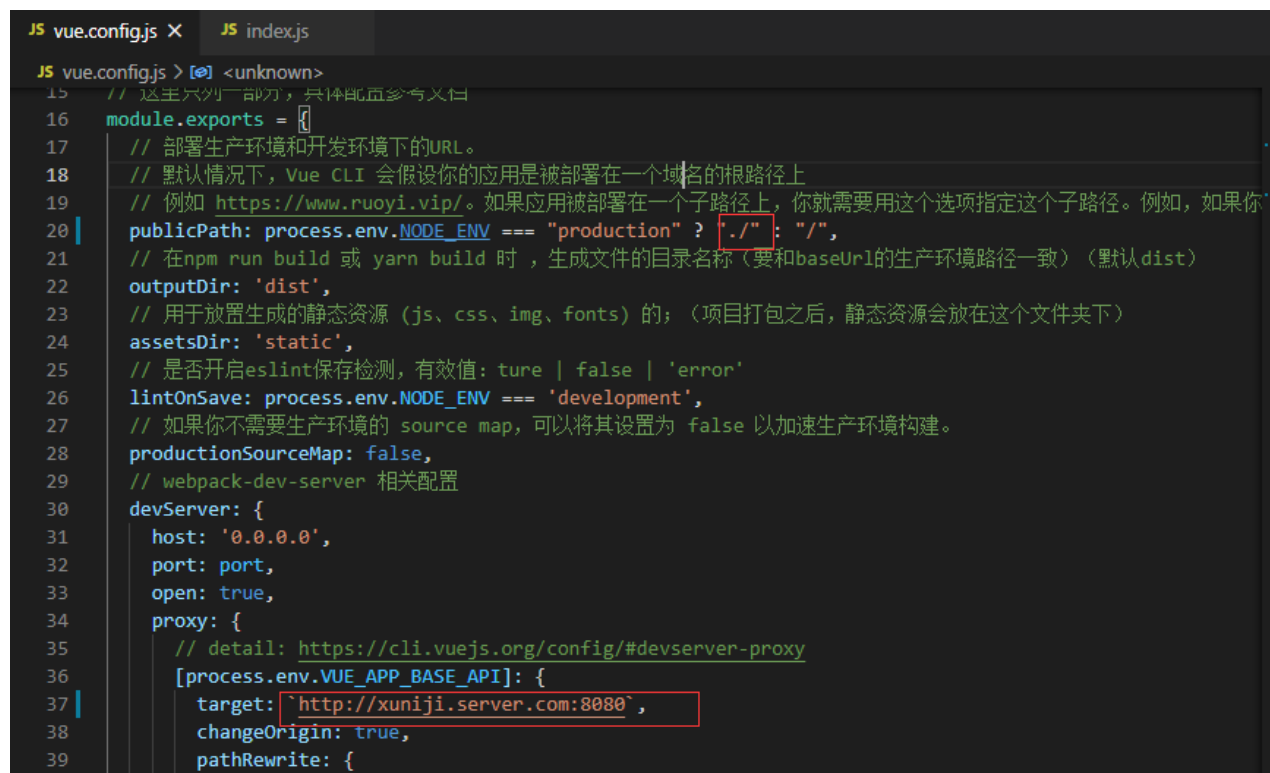
四.config机器前端部署

1.在config机器上安装jdk1.8,mysql, redis, nacos, nginx安装流程我就不一一解释了，大家可自行百度，后面我会贴上必要的配置信息

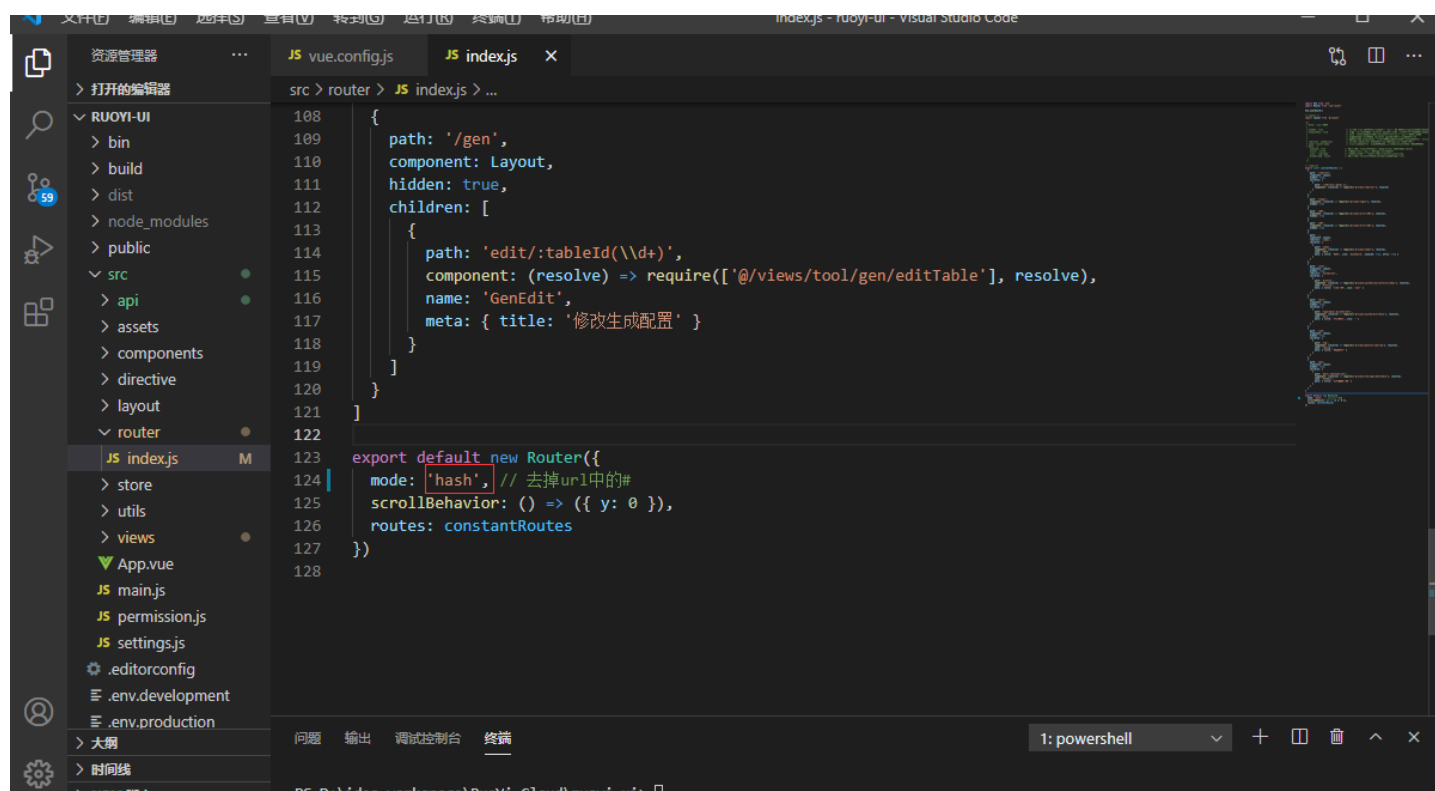
2. 修改前端的vue.config.js的配置

配置修改：前端ui文件中的index.js文件、vue.config.js文件

如下图：



```
JS vue.config.js X JS index.js
JS vue.config.js > [?] <unknown>
15 // 这里只列出一部分，具体配置参考文档
16 module.exports = {
17   // 部署生产环境和开发环境下的URL。
18   // 默认情况下，Vue CLI 会假设你的应用是被部署在一个域名的根路径上
19   // 例如 https://www.ruoyi.vip/。如果应用被部署在一个子路径上，你就需要用这个选项指定这个子路径。例如，如果你
20   // 在npm run build 或 yarn build 时，生成文件的目录名称（要和baseUrl的生产环境路径一致）（默认dist）
21   // 在npm run build 或 yarn build 时，生成文件的目录名称（要和baseUrl的生产环境路径一致）（默认dist）
22   outputDir: 'dist',
23   // 用于放置生成的静态资源（js、css、img、fonts）的；（项目打包之后，静态资源会放在这个文件夹下）
24   assetsDir: 'static',
25   // 是否开启eslint保存检测，有效值：ture | false | 'error'
26   lintOnSave: process.env.NODE_ENV === 'development',
27   // 如果你不需要生产环境的 source map，可以将其设置为 false 以加速生产环境构建。
28   productionSourceMap: false,
29   // webpack-dev-server 相关配置
30   devServer: {
31     host: '0.0.0.0',
32     port: port,
33     open: true,
34     proxy: {
35       // detail: https://cli.vuejs.org/config/#devserver-proxy
36       [process.env.VUE_APP_BASE_API]: {
37         target: 'http://xuniji.server.com:8080',
38         changeOrigin: true,
39         pathRewrite: {
```



```
src > router > JS indexjs > ...
108 {
109   path: '/gen',
110   component: Layout,
111   hidden: true,
112   children: [
113     {
114       path: 'edit/:tableId(\\d+)',
115       component: (resolve) => require(['@/views/tool/gen/editTable'], resolve),
116       name: 'GenEdit',
117       meta: { title: '修改生成配置' }
118     }
119   ]
120 }
121 ]
122 }
123 export default new Router({
124   mode: 'hash', // 去掉url中的#
125   scrollBehavior: () => ({ y: 0 }),
126   routes: constantRoutes
127 })
128
```

3. 修改后在前端执行 npm run build:prod 生成dist包文件，放到config服务器的/home/ruoyi/projects/ruoyi-ui目录下

4. nginx.conf配置

```
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    include mime.types;
    default_type application/octet-stream;

    sendfile on;

    keepalive_timeout 65;

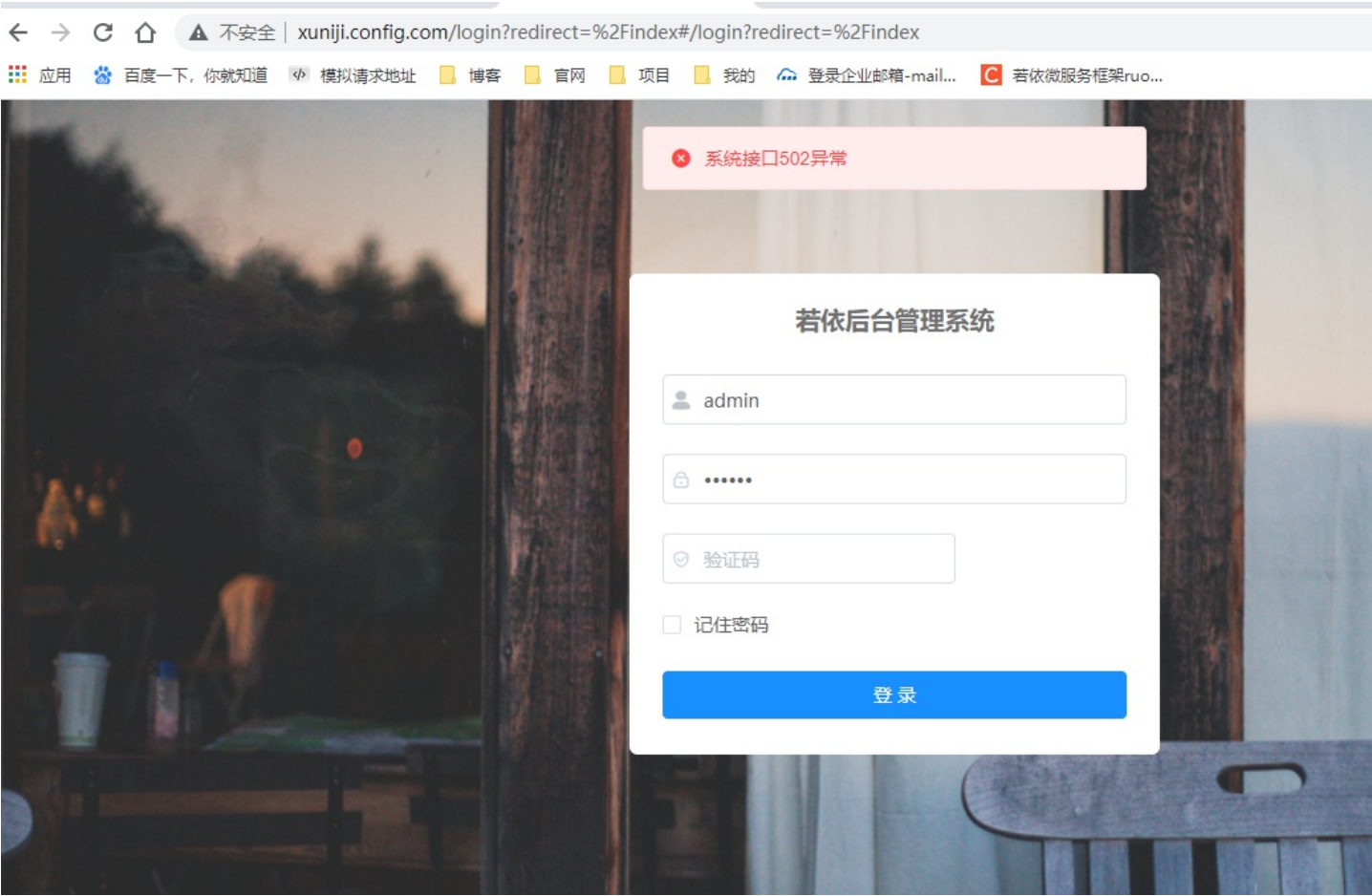
    server {
        listen 80;
        server_name localhost;

        location / {
            root /home/ruoyi/projects/ruoyi-ui;
            try_files $uri $uri/ /index.html;
            index index.html index.htm;
        }

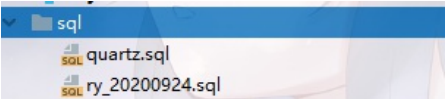
        error_page 500 502 503 504 /50x.html;
        location = /50x.html {
            root html;
        }

        location /prod-api/{
            proxy_set_header Host $http_host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header REMOTE-HOST $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_pass http://xuniji.server.com:8080/;
        }
    }
}
```

5.启动nginx
浏览器访问xuniji.config.com出现下图的效果则前端代码部署好了



6.将三个sql导入到mysql中



7.修改nacos的application.properties配置

```
xuniji.config.com x xuniji.server.com
##### Spring Boot Related Configurations #####
### Default web context path:
server.servlet.contextPath=/nacos
### Default web server port:
server.port=8848

##### Network Related Configurations #####
### If prefer hostname over ip for Nacos server addresses in cluster.conf:
# nacos.inetutils.prefer-hostname-over-ip=false

### Specify local server's IP:
# nacos.inetutils.ip-address=

##### Config Module Related Configurations #####
### If user MySQL as datasource:
spring.datasource.platform=mysql

### Count of DB:
db.num=1

### Connect URL of DB:
db.url.0=jdbc:mysql://xuniji.config.com:3306/ry-config?characterEncoding=utf8&connectTimeout=1000&socketTimeout=3000&autoReconnect=true
db.user=root
db.password=123456

##### Naming Module Related Configurations #####
### Data dispatch task execution period in milliseconds:
# nacos.naming.distro.taskDispatchPeriod=200

### Data count of batch sync task:
# nacos.naming.distro.batchSyncKeyCount=1000

### Retry delay in milliseconds if sync task failed:
# nacos.naming.distro.syncRetryDelay=5000

### If enable data warmup. If set to false, the server would accept request without local data preparation:
# nacos.naming.data.warmup=true

### If enable the instance auto expiration, kind like of health check of instance:
# nacos.naming.expireInstance=true
```

1,1 页

8.统一启动redis, nacos,nginx的shell脚本

```
cd /usr/local
```

```
vim start.sh
```

```
#!/bin/bash

echo "启动redis"
/usr/local/redis/bin/redis-server /usr/local/redis/conf/redis.conf

echo "启动nacos"
/usr/local/nacos/bin/startup.sh -m standalone

echo "启动nginx"
/usr/local/nginx/sbin/nginx
```

```
chmod u+x start.sh
```

```
./start.sh
```

9.修改nacos配置信息

修改redis的ip

```
1 spring:
2   redis:
3     host: xuniji.config.com
4     port: 6379
```

修改mysql的ip

Beta发布: ☐ 默认不要勾选。

配置格式: ☐ TEXT ☐ JSON ☐ XML ☒ YAML ☐ HTML ☐ Properties

配置内容 ? :

```
1 spring:
2   datasource:
3     driver-class-name: com.mysql.cj.jdbc.Driver
4     url: jdbc:mysql://xuniji.config.com:3306/ry-cloud?useUnicode=true&characterEncoding=utf8&
5       zeroDateTimeBehavior=convertToNull&useSSL=true&serverTimezone=GMT%2B8
6     username: root
7     password: 123456
8   redis:
9     host: xuniji.config.com
10    port: 6379
11    password:
```

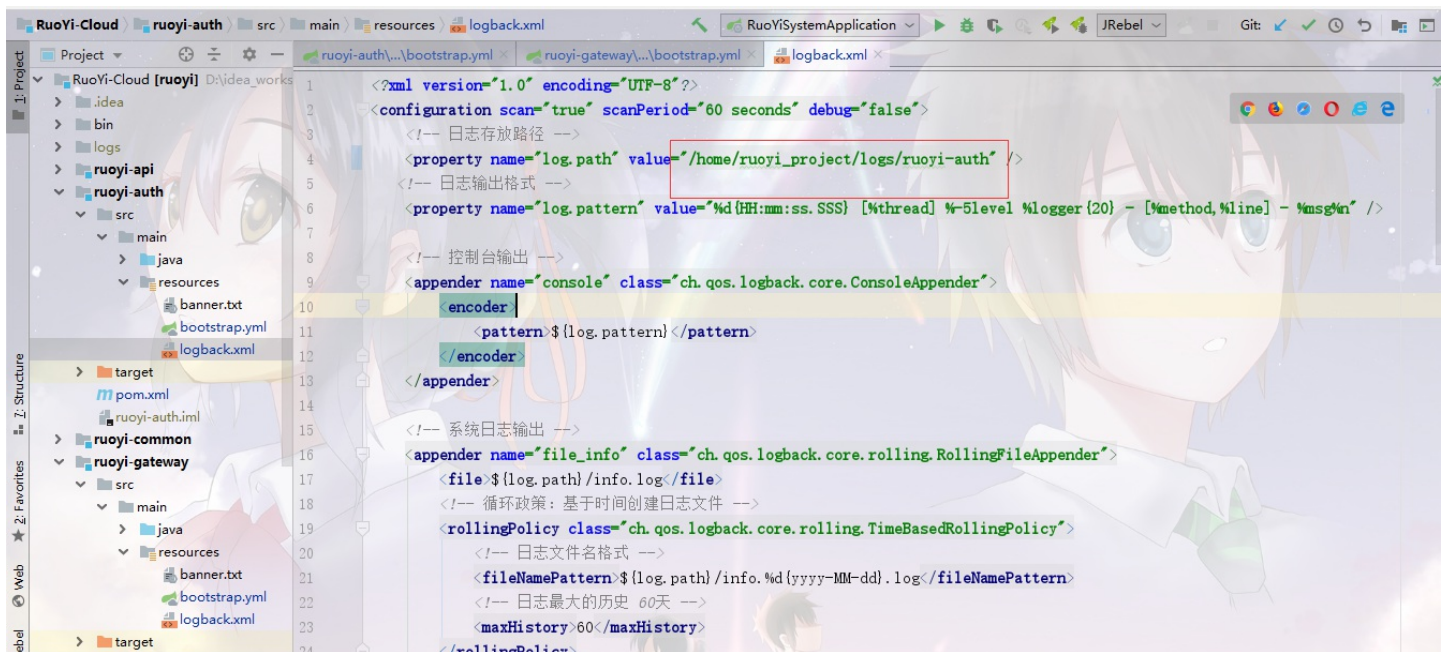
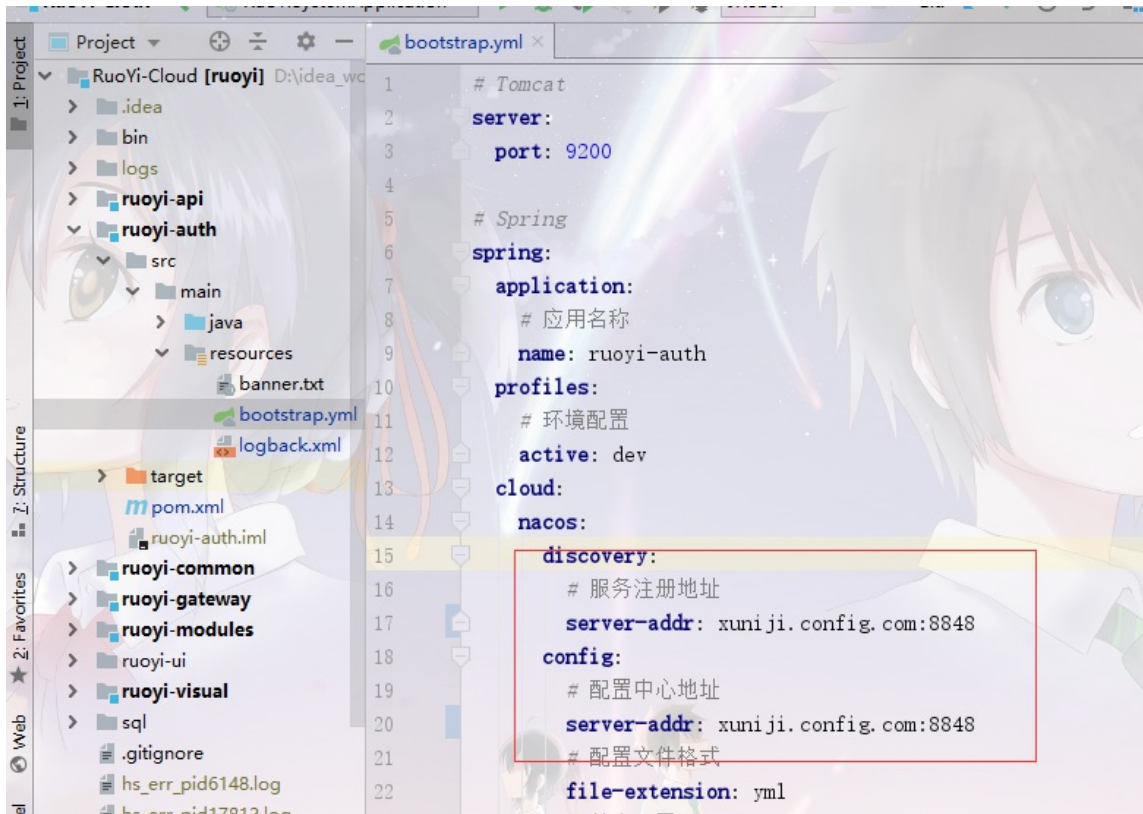
其余的服务也是一样，依次检查修改

五.server机器后端部署

1.在server服务器上安装jdk1.8

2.修改项目的配置信息

以ruoyi-auth为例



修改pom.xml文件，将build替换成下面的内容

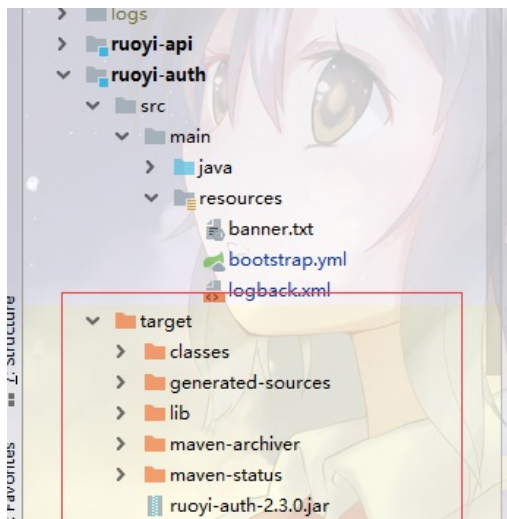
```

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <executions>
        <execution>
          <goals>
            <goal>repackage</goal>
          </goals>
        </execution>
      </executions>
    </plugin>
    <plugin>
      <!-- 打包时去除第三方依赖 -->
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <layout>ZIP</layout>
        <includes>
          <include>
            <groupId>non-exists</groupId>
            <artifactId>non-exists</artifactId>
          </include>
        </includes>
      </configuration>
    </plugin>
  <!-- 拷贝第三方依赖文件到指定目录 -->
  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-dependency-plugin</artifactId>
    <executions>
      <execution>
        <id>copy-dependencies</id>
        <phase>package</phase>
        <goals>
          <goal>copy-dependencies</goal>
        </goals>
        <configuration>
          <!-- target/lib是依赖jar包的输出目录， 根据自己喜好配置 -->
          <outputDirectory>target/lib</outputDirectory>
          <excludeTransitive>false</excludeTransitive>
          <stripVersion>false</stripVersion>
          <includeScope>runtime</includeScope>
        </configuration>
      </execution>
    </executions>
  </plugin>
</plugins>
</build>

```

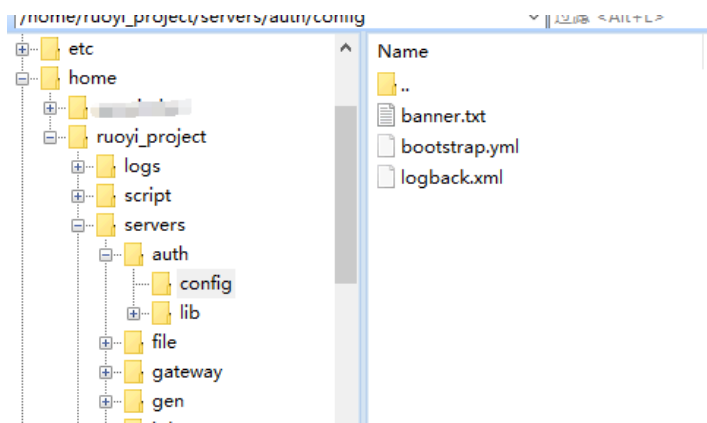
其余的服务也是一样，依次检查修改

3.idea打包项目

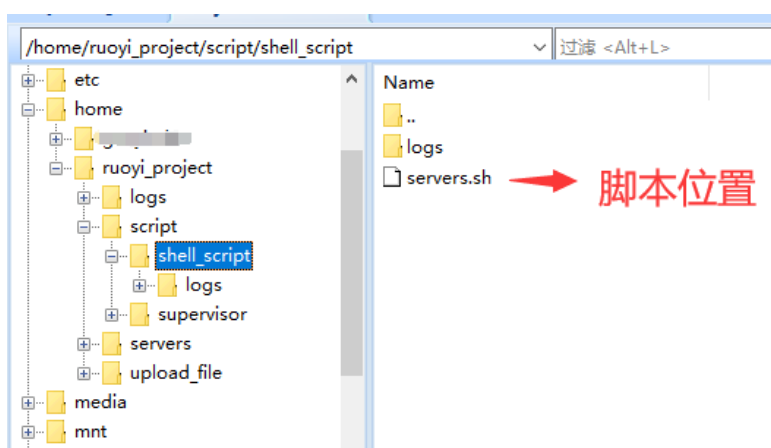


4.按照成品展示的第四步将文件夹创建好后开始上传jar包和依赖包，以及配置文件

配置文件我上传了这些



5.shell脚本管理jar包开启关闭(不推荐,shell脚本学习的话可以试一试)，test服务是我自己创建的一个服务，请忽略



```
#!/bin/bash
CURRENT_PATH="/home/ruoyi_project/servers/"
LOG_PATH="/home/ruoyi_project/script/shell_script/logs/"

SERVER_NAMES=("auth" "gateway" "file" "gen" "job" "system" "test" "monitor")
JAR=""
PID=""

#根据不同的服务定义项目路径和日志路径
```

```

if [[ "${SERVER_NAMES[@]}" =~ "${2}" ]]; then
    CURRENT_PATH=${CURRENT_PATH}${2}/
    LOG_PATH=${LOG_PATH}${2}.log
    JAR=$(find $CURRENT_PATH -maxdepth 1 -name "*.jar")
    PID=$(ps -ef | grep $JAR | grep -v grep | awk '{ print $2 }')
elif [[ "${2}" != "all" ]]; then
    echo -e "Usage: ./servers.sh ${1} {auth|gateway|file|gen|job|system|test|monitor|all} \n" >&2
    exit 1
fi

#服务操作的通用方法
function call_servers(){
    for server_name in ${SERVER_NAMES[@]}
    do
        $1 $2 ${server_name}
    done
}

case "${1}" in
    "start")
        if [ "${2}" == "all" ];then
            call_servers ${0} ${1}
        elif [ ! -z "$PID" ]; then
            echo "$JAR 已经启动， 进程号: $PID "
        else
            echo -e "启动 $JAR ... \n"
            cd $CURRENT_PATH
            nohup java -Dloader.path=$CURRENT_PATH,resources,lib -Dfile.encoding=UTF-8 -jar $JAR >$LOG_PATH 2>&1 &
            if [ "$?"="0" ]; then
                echo -e "$JAR 启动完成， 请查看日志确保成功 \n"
            else
                echo -e "$JAR 启动失败 \n"
            fi
        fi
        ;;
    "stop")
        if [ "${2}" == "all" ];then
            call_servers ${0} ${1}
        elif [ -z "$PID" ]; then
            echo -e "$JAR 没有在运行， 无需关闭 \n"
        else
            echo -e "关闭 $JAR ..."
            kill -9 $PID
            if [ "$?"="0" ]; then
                echo -e "服务已关闭 \n"
            else
                echo -e "服务关闭失败 \n"
            fi
        fi
        ;;
    "restart")
        if [ "${2}" == "all" ];then
            call_servers ${0} ${1}
        else
            ${0} stop ${2}
            ${0} start ${2}
        fi
        ;;
    "kill")
        if [ "${2}" == "all" ];then
            call_servers ${0} ${1}
        else
            echo -e "强制关闭 $JAR \n"
            killall $JAR
            if [ "$?"="0" ]; then

```

```

        echo -e "成功 \n"
    else
        echo -e "失败 \n"
    fi
fi
;;
"status")
if [ "${2}" == "all" ];then
    call_servers ${0} ${1}
elif [ ! -z "$PID" ]; then
    echo -e "$JAR 正在运行 \n"
else
    echo -e "$JAR 未在运行 \n"
fi
;;
*)
echo -e "Usage: ./springboot.sh {start|stop|restart|status|kill} ${2} \n" >&2
exit 1
esac

```

6.脚本效果展示

```

[root@xuniji shell_script]# ./servers.sh status auth
/home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar 未在运行

[root@xuniji shell_script]# ./servers.sh status all
/home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar 未在运行

/home/ruoyi_project/servers/gateway/ruoyi-gateway-2.3.0.jar 未在运行

/home/ruoyi_project/servers/file/ruoyi-modules-file-2.3.0.jar 未在运行

/home/ruoyi_project/servers/gen/ruoyi-modules-gen-2.3.0.jar 未在运行

/home/ruoyi_project/servers/job/ruoyi-modules-job-2.3.0.jar 未在运行

/home/ruoyi_project/servers/system/ruoyi-modules-system-2.3.0.jar 未在运行

/home/ruoyi_project/servers/test/ruoyi-modules-test-2.3.0.jar 未在运行

/home/ruoyi_project/servers/monitor/ruoyi-visual-monitor-2.3.0.jar 未在运行

[root@xuniji shell_script]# ./servers.sh start auth
启动 /home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar ...

/home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar 启动完成，请查看日志确保成功

[root@xuniji shell_script]# ./servers.sh stop auth
关闭 /home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar ...
服务已关闭

[root@xuniji shell_script]# ./servers.sh start all
启动 /home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar ...

/home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar 启动完成，请查看日志确保成功

启动 /home/ruoyi_project/servers/gateway/ruoyi-gateway-2.3.0.jar ...

/home/ruoyi_project/servers/gateway/ruoyi-gateway-2.3.0.jar 启动完成，请查看日志确保成功

启动 /home/ruoyi_project/servers/file/ruoyi-modules-file-2.3.0.jar ...

/home/ruoyi_project/servers/file/ruoyi-modules-file-2.3.0.jar 启动完成，请查看日志确保成功

启动 /home/ruoyi_project/servers/gen/ruoyi-modules-gen-2.3.0.jar ...

```

```
/home/ruoyi_project/servers/gen/ruoyi-modules-gen-2.3.0.jar 启动完成，请查看日志确保成功

启动 /home/ruoyi_project/servers/job/ruoyi-modules-job-2.3.0.jar ...

/home/ruoyi_project/servers/job/ruoyi-modules-job-2.3.0.jar 启动完成，请查看日志确保成功

启动 /home/ruoyi_project/servers/system/ruoyi-modules-system-2.3.0.jar ...

/home/ruoyi_project/servers/system/ruoyi-modules-system-2.3.0.jar 启动完成，请查看日志确保成功

启动 /home/ruoyi_project/servers/test/ruoyi-modules-test-2.3.0.jar ...

/home/ruoyi_project/servers/test/ruoyi-modules-test-2.3.0.jar 启动完成，请查看日志确保成功

启动 /home/ruoyi_project/servers/monitor/ruoyi-visual-monitor-2.3.0.jar ...

/home/ruoyi_project/servers/monitor/ruoyi-visual-monitor-2.3.0.jar 启动完成，请查看日志确保成功

[root@xuniji shell_script]# ./servers.sh status all
/home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar 正在运行

/home/ruoyi_project/servers/gateway/ruoyi-gateway-2.3.0.jar 正在运行

/home/ruoyi_project/servers/file/ruoyi-modules-file-2.3.0.jar 正在运行

/home/ruoyi_project/servers/gen/ruoyi-modules-gen-2.3.0.jar 正在运行

/home/ruoyi_project/servers/job/ruoyi-modules-job-2.3.0.jar 正在运行

/home/ruoyi_project/servers/system/ruoyi-modules-system-2.3.0.jar 正在运行

/home/ruoyi_project/servers/test/ruoyi-modules-test-2.3.0.jar 正在运行

/home/ruoyi_project/servers/monitor/ruoyi-visual-monitor-2.3.0.jar 正在运行

[root@xuniji shell_script]# ./servers.sh stop all
关闭 /home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar ...
服务已关闭

关闭 /home/ruoyi_project/servers/gateway/ruoyi-gateway-2.3.0.jar ...
服务已关闭

关闭 /home/ruoyi_project/servers/file/ruoyi-modules-file-2.3.0.jar ...
服务已关闭

关闭 /home/ruoyi_project/servers/gen/ruoyi-modules-gen-2.3.0.jar ...
服务已关闭

关闭 /home/ruoyi_project/servers/job/ruoyi-modules-job-2.3.0.jar ...
服务已关闭

关闭 /home/ruoyi_project/servers/system/ruoyi-modules-system-2.3.0.jar ...
服务已关闭

关闭 /home/ruoyi_project/servers/test/ruoyi-modules-test-2.3.0.jar ...
服务已关闭

关闭 /home/ruoyi_project/servers/monitor/ruoyi-visual-monitor-2.3.0.jar ...
服务已关闭

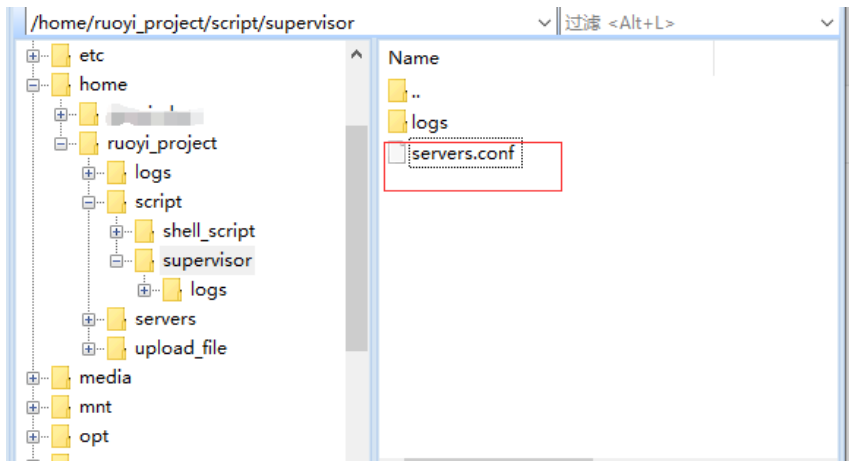
[root@xuniji shell_script]# ./servers.sh status all
/home/ruoyi_project/servers/auth/ruoyi-auth-2.3.0.jar 未在运行

/home/ruoyi_project/servers/gateway/ruoyi-gateway-2.3.0.jar 未在运行
```



```
home/ruoyi_project/servers/file/ruoyi-modules-file-2.3.0.jar 未在运行
home/ruoyi_project/servers/gen/ruoyi-modules-gen-2.3.0.jar 未在运行
home/ruoyi_project/servers/job/ruoyi-modules-job-2.3.0.jar 未在运行
home/ruoyi_project/servers/system/ruoyi-modules-system-2.3.0.jar 未在运行
home/ruoyi_project/servers/test/ruoyi-modules-test-2.3.0.jar 未在运行
home/ruoyi_project/servers/monitor/ruoyi-visual-monitor-2.3.0.jar 未在运行
```

7.通过supervisor来管理服务状态



(1) 安装守护程序supervisord(推荐)

```
yum install python-setuptools
easy_install supervisor
echo_supervisord_conf > /etc/supervisord.conf
```

(2) 修改配置文件

```
vim /etc/supervisord.conf
```

```
文件(F) 编辑(E) 查看(V) 选项(O) 传输(T) 脚本(S) 工具(L) 窗口(W) 帮助(H)
输入主机 <Alt+R>
xuniji.config.com xuniji.server.com x
;numprocs=1           ; number of processes copies to start (def 1)
;events=EVENT         ; event notif. types to subscribe to (req'd)
;buffer_size=10       ; event buffer queue size (default 10)
;directory=/tmp       ; directory to cwd to before exec (def no cwd)
;umask=022            ; umask for process (default None)
;priority=-1          ; the relative start priority (default -1)
;autostart=true       ; start at supervisord start (default: true)
;startsecs=1         ; # of secs prog must stay up to be running (def. 1)
;startretries=3       ; max # of serial start failures when starting (default 3)
;autorestart=unexpected ; autorestart if exited after running (def: unexpected)
;exitcodes=0         ; 'expected' exit codes used with autorestart (default 0)
;stopsignal=QUIT      ; signal used to kill process (default TERM)
;stopwaitsecs=10      ; max num secs to wait b4 SIGKILL (default 10)
;stopasgroup=false    ; send stop signal to the UNIX process group (default false)
;killasgroup=false    ; SIGKILL the UNIX process group (def false)
;user=chris           ; setuid to this UNIX account to run the program
;redirect_stderr=false ; redirect_stderr=true is not allowed for eventlisteners
;stdout_logfile=/a/path ; stdout log path, NONE for none; default AUTO
;stdout_logfile_maxbytes=1MB ; max # logfile bytes b4 rotation (default 50MB)
;stdout_logfile_backups=10 ; # of stdout logfile backups (0 means none, default 10)
;stdout_events_enabled=false ; emit events on stdout writes (default false)
;stderr_logfile=/a/path ; stderr log path, NONE for none; default AUTO
;stderr_logfile_maxbytes=1MB ; max # logfile bytes b4 rotation (default 50MB)
;stderr_logfile_backups=10 ; # of stderr logfile backups (0 means none, default 10)
;stderr_events_enabled=false ; emit events on stderr writes (default false)
;stderr_syslog=false   ; send stderr to syslog with process name (default false)
;environment=A="1",B="2" ; process environment additions
;serverurl=AUTO        ; override serverurl computation (childutils)

; The sample group section below shows all possible group values. Create one
; or more 'real' group: sections to create "heterogeneous" process groups.

[group:thegroupname]
;programs=programe1,programe2 ; each refers to 'x' in [program:x] definitions
;priority=999                ; the relative start priority (default 999)

; The [include] section can just contain the "files" setting. This
; setting can list multiple files (separated by whitespace or
; newlines). It can also contain wildcards. The filenames are
; interpreted as relative to this file. Included files *cannot*
; include files themselves.

[include]
files=/home/ruoyi_project/script/supervisor/servers.conf
/etc/supervisord.conf 170L, 10630C

166,1 底端
```

(3)添加需要守护的程序到servers.conf

```
vim /home/ruoyi_project/script/supervisor/servers.conf
```

```
[program:auth]
;需要执行的命令
command=java -Dloader.path=/home/ruoyi_project/servers/auth/lib/resources/lib -Dfile.encoding=UTF-8 -jar /home/ruoyi_project/server
s/auth/ruoyi-auth-2.3.0.jar
;命令执行的目录
directory=/home/ruoyi_project/servers/auth
;用户
user=root
stopsignal=INT
;是否启动
autostart=true
;是否自动重启
autorestart=true
;自动重启时间间隔 (s)
startsecs=3
;错误日志文件
stderr_logfile=/home/ruoyi_project/script/supervisor/logs/auth_err.log
;输出日志文件
stdout_logfile=/home/ruoyi_project/script/supervisor/logs/auth_info.log

[program:gateway]
command=java -Dloader.path=/home/ruoyi_project/servers/gateway/lib/resources/lib -Dfile.encoding=UTF-8 -jar /home/ruoyi_project/se
rvers/gateway/ruoyi-gateway-2.3.0.jar
directory=/home/ruoyi_project/servers/gateway
user=root
stopsignal=INT
autostart=true
```

```
autorestart=true
startsecs=3
stderr_logfile=/home/ruoyi_project/script/supervisor/logs/gateway_err.log
stdout_logfile=/home/ruoyi_project/script/supervisor/logs/gateway_info.log

[program:file]
command=java -Dloader.path=java -Dloader.path=/home/ruoyi_project/servers/file/lib/resources/lib -Dfile.encoding=UTF-8 -jar /home/ruoyi_project/servers
/file/ruoyi-modules-file-2.3.0.jar
directory=/home/ruoyi_project/servers/file
user=root
stopsignal=INT
autostart=true
autorestart=true
startsecs=3
stderr_logfile=/home/ruoyi_project/script/supervisor/logs/file_err.log
stdout_logfile=/home/ruoyi_project/script/supervisor/logs/file_info.log

[program:gen]
command=java -Dloader.path=java -Dloader.path=/home/ruoyi_project/servers/gen/lib/resources/lib -Dfile.encoding=UTF-8 -jar /home/ruoyi_project/server
s/gen/ruoyi-modules-gen-2.3.0.jar
directory=/home/ruoyi_project/servers/gen
user=root
stopsignal=INT
autostart=true
autorestart=true
startsecs=3
stderr_logfile=/home/ruoyi_project/script/supervisor/logs/gen_err.log
stdout_logfile=/home/ruoyi_project/script/supervisor/logs/gen_info.log

[program:job]
command=java -Dloader.path=java -Dloader.path=/home/ruoyi_project/servers/job/lib/resources/lib -Dfile.encoding=UTF-8 -jar /home/ruoyi_project/servers
/job/ruoyi-modules-job-2.3.0.jar
directory=/home/ruoyi_project/servers/job
user=root
stopsignal=INT
autostart=true
autorestart=true
startsecs=3
stderr_logfile=/home/ruoyi_project/script/supervisor/logs/job_err.log
stdout_logfile=/home/ruoyi_project/script/supervisor/logs/job_info.log

[program:system]
command=java -Dloader.path=java -Dloader.path=/home/ruoyi_project/servers/system/lib/resources/lib -Dfile.encoding=UTF-8 -jar /home/ruoyi_project/ser
vers/system/ruoyi-modules-system-2.3.0.jar
directory=/home/ruoyi_project/servers/system
user=root
stopsignal=INT
autostart=true
autorestart=true
startsecs=3
stderr_logfile=/home/ruoyi_project/script/supervisor/logs/system_err.log
stdout_logfile=/home/ruoyi_project/script/supervisor/logs/system_info.log

[program:monitor]
command=java -Dloader.path=java -Dloader.path=/home/ruoyi_project/servers/monitor/lib/resources/lib -Dfile.encoding=UTF-8 -jar /home/ruoyi_project/ser
vers/monitor/ruoyi-visual-monitor-2.3.0.jar
directory=/home/ruoyi_project/servers/monitor
user=root
stopsignal=INT
autostart=true
autorestart=true
startsecs=3
```

```
stderr_logfile=/home/ruoyi_project/script/supervisor/logs/monitor_err.log
stdout_logfile=/home/ruoyi_project/script/supervisor/logs/monitor_info.log
```

(4)设置supervisor开机启动,进入目录/usr/lib/systemd/system/新增文件supervisord.service，内容如下：

```
[Unit]
Description=Supervisor daemon

[Service]
Type=forking
ExecStart=/usr/bin/supervisord -c /etc/supervisord.conf
ExecStop=/usr/bin/supervisorctl shutdown
ExecReload=/usr/bin/supervisorctl reload
KillMode=process
Restart=on-failure
RestartSec=42s

[Install]
WantedBy=multi-user.target
```

执行

```
systemctl enable supervisord.service
```

8.supervisor效果展示

```
[root@xuniji script]# supervisorctl status
auth                RUNNING   pid 33892, uptime 0:00:12
file                RUNNING   pid 33283, uptime 0:04:16
gateway             RUNNING   pid 33281, uptime 0:04:16
gen                 RUNNING   pid 33292, uptime 0:04:16
job                 RUNNING   pid 33282, uptime 0:04:16
monitor             RUNNING   pid 33278, uptime 0:04:16
system              RUNNING   pid 33279, uptime 0:04:16
test                RUNNING   pid 33284, uptime 0:04:16
[root@xuniji script]# supervisorctl stop auth
auth: stopped
[root@xuniji script]# supervisorctl start auth
auth: started
[root@xuniji script]# supervisorctl stop all
monitor: stopped
auth: stopped
gateway: stopped
test: stopped
gen: stopped
job: stopped
file: stopped
system: stopped
```

supervisorctl stop all 停止全部

supervisorctl stop program_name # 停止某一个进程，program_name 为 [program:x] 里的x

supervisorctl start program_name # 启动个进程

supervisorctl restart program_name # 重启某个进程

supervisorctl update 修改配置文件后更新配置

END(以上只是个人的一些部署想法和方式，如果大家有更好的方式，欢迎提出来，互相学习)